

Data Structure and Programming, Spring 2025
programming assignment #3
Binary Search Tree

Pei-Yuan, Wu peiyuanwu@ntu.edu.tw
Yi- Chou Lin b10901149@ntu.edu.tw
Chiao-Yun Chin b10901154@ntu.edu.tw
Yi-Chen Wang b10901143@ntu.edu.tw

RELEASE DATE: 12:00, 04/24/2025
DUE DATE: 23:59, 05/08/2025

Late policy: $Score = Score \times 0.7^{(h/24)}$, $h = \text{late hours} \leq 48$

1 Problem Description

In this homework, you're going to implement a binary search tree by yourself. Your BSTree needs to have the following functions: *insert()*, *delete()*, *find_max()*, *find_min()*, *inorder()*, *preorder()*, *postorder()*, *level()*, *leafnode()*, *internalnode()*.

There are more detailed descriptions of the functions.

insert()

The function follows the rule of the standard binary search tree. The key in each node must be greater than any key stored in the left sub-tree, and less than any key stored in the right sub-tree.

delete()

The function removes a node whose key equals the input value. Please maintain the properties of binary search tree after doing *delete()* operation. If the given key value is not in the current tree, don't modify the tree. (If there are two children, replacing the node's value with its smallest value in the right sub-tree)

find_max()

The function finds the maximum value in the current binary search tree.

find_min()

The function finds the minimum value in the current binary search tree.

inorder()

Print the current tree in the inorder traversal sequence. You should use space(' ') to separate the key of each node, and the output should end up with a newline character.

preorder()

Print the current tree in the preorder traversal sequence. You should use space(' ') to separate the key of each node, and the output should end up with a newline character.

postorder()

Print the current tree in the postorder traversal sequence. You should use space(' ') to separate the key of each node, and the output should end up with a newline character.

level()

Print the height of the current tree. If the tree is empty, the height of the tree is 0. (The height of the leaf node is 0)

leafnode()

Print all leaf nodes of the current tree from the leftmost one to the rightmost one. You should use space(' ') to separate the key of each node, and the output should end up with a newline character.

internalnode()

Print all internal nodes of the current tree from the smallest one to the largest one. You should use space(' ') to separate the key of each node, and the output should end up with a newline character.

2 Input/Output Specification

We have done the input function for you. You only need to focus on the functions above.

Here is an input/output example:

Input:	Output:
insert 50 10 15 20 100 55 80 250	20 15 10 80 55 250 100 50
postorder	10 15 50 55 100
Internalnode	50 10 20 100 55 250
delete 80	250
Delete 15	10
Preorder	2
Max	20 55 250
Min	10 20 55 100 250
Level	
Leafnode	
delete 50	
Inorder	

Figure 1: The input and output formats.

3 Evaluation

We have provided a code file **BSTree.py**. You have to fill in the class **BSTree.py** which is used for testing. Write your codes in **TODO**.

1. Do not modify the interface of the functions, but you can add your own functions.
2. Binary search tree is not always balanced.
3. Use command "python3 BSTree.py --input input1.txt --output output1.txt" to check your output. Use command "bash evaluation.sh" to check result (just as P1).

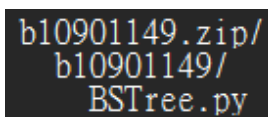
4 Grading policy

This programming assignment will be graded based on both correctness and complexity. There are 10 test cases in total: 3 public test cases, 5 private test cases, and 2 complexity test cases. Each test case contributes 10% to your final score, for a total of 100%. The public test cases come with their corresponding golden outputs and can be evaluated using the provided script evaluation.sh. The private test cases are similar in size, but may include edge cases such as unbalanced binary search trees. The complexity test cases are designed to assess the efficiency of your implementation. Please ensure that your traversal algorithm runs in $O(n)$ time, and both insertion and deletion operations achieve $O(\log n)$ time complexity in the average case.

5 Submission

Please put your codes (including BSTree.py or any other code files) into a directory named **studentID** and compress the directory into studentID.zip to NTUCOOL. The homework is due on **05/08**, at **23:59**

- Please make sure your code can compile. You **CANNOT** ask TA to debug or modify your code for evaluation.
- We will unzip your submission by running `unzip *.zip`. Your submitted folder structure should be: **(20% penalty for invalid format)**



```
b10901149.zip/  
b10901149/  
BSTree.py
```

Figure 2: Submission format.